Distributed DAG Task Scheduler: MS3 Specification

This specification defines the behavior we require from the MS3 scheduler. It covers the system model, safety and liveness properties, supported failures, and performance measurements. The MS1 statements are reproduced unchanged; the text following each statement describes the implemented MS3 contract.

For test names and property coverage, see test-matrix.md. For results and observed limitations, see ms3-progress-report.md. The request and event formats are documented in api.md and evidence.md.

1. Overview

Original MS1 statement:

"We are building a fault-tolerant distributed task scheduler for a video-processing platform. Each uploaded video creates a directed acyclic graph (DAG) of tasks such as transcoding, thumbnail generation, subtitle generation, and publishing. Some tasks can run in parallel, while others must wait for their dependencies. The system consists of one scheduler and multiple worker nodes. The scheduler tracks dependencies and assigns ready tasks to available workers. Workers execute tasks and report completion. If a worker fails before finishing, the unfinished task can be reassigned. We are not building a full video-sharing platform, authentication, billing, or production-quality video-processing algorithms."

A client submits a JSON manifest containing the tasks of a DAG and their relationships. The scheduler accepts operations from a fixed allowlist. It makes a task ready only after all its dependencies have succeeded and marks the job complete after every task has succeeded.

The scheduler is one Java 21 process. It checks incoming jobs, tracks which tasks are ready, assigns them in FIFO order, and decides which results to accept. It keeps its state in memory and uses one mutex to protect updates.

Workers are Java processes that poll the scheduler over HTTP/JSON. Each worker handles one task at a time: it acknowledges the assignment, runs the operation, uploads the output, and reports the result. Heartbeats keep its assignment active. Workers do not need an inbound port.

The artifact store is one Java process that serves immutable, content-verified files. The client, either a CLI or the Python harness, submits jobs and retrieves their results. The scheduler also checks outputs in the store.

For the video workload, `inspect` runs first. The two transcodes, thumbnail creation, and fixture-subtitle copy then run in parallel, followed by `publish`, which writes an output manifest. This is a reproducible test workload, not a production media-processing service.

The scheduler retains its decision history in memory. This allows tests to inspect the order of scheduling and completion events, but it also limits the length of a run. The memory probe measured about 10 KB retained per completed task. A 128 MiB heap therefore suggests a capacity near 11,000 tasks, not a guaranteed maximum. See the progress report for the probe results.

We do not provide scheduler replication, durable scheduler state, or failover. Artifact-store recovery and replication, authentication, authorization, a web interface, priorities, cancellation, deadlines, retry limits, real speech recognition, and production media processing are also outside MS3. We assume

workers are honest. The system can execute a task more than once, but it must accept no more than one completion for that task. It does not guarantee exactly-once external side effects.

2. System model

Requests and replies can be lost, delayed, duplicated, or reordered. We assume fair-loss communication: repeated retries eventually get through. Operations that change state carry identifiers, allowing a repeated request to be recognized rather than applied again.

A worker may crash, restart, pause, run slowly, or lose communication. A restarted worker joins with a new session and cannot reuse its old attempts. The scheduler is assumed to remain running throughout a job's execution. If it restarts, it creates a new, empty run; requests from an earlier run receive `409 STALE_RUN`. The artifact store is assumed to remain available and correct, with published objects staying unchanged.

Safety does not require synchronized clocks. The scheduler makes lease decisions using its own monotonic clock. Progress requires a running scheduler and a worker that can reach the scheduler and artifact store long enough to complete an operation.

Each assignment has a lease. `LEASE_TIMEOUT_MS` defaults to 5000 ms. The assignment grants the lease; an accepted start acknowledgment and each heartbeat renew it. Workers obtain the lease duration from `/v1/health` and normally wait `lease/5` between heartbeat requests. The wait begins after the previous request finishes, so slow requests reduce the time left for renewal. With prompt requests, the default lease can accommodate three consecutive lost heartbeats, but this is not an unconditional guarantee.

When an attempt expires, its task goes back to the end of the ready queue. The next assignment gets a higher attempt number. The scheduler checks that number whenever it receives a start, heartbeat, or report. For example, if attempt 1 expires and attempt 2 is assigned, a late report from attempt 1 cannot change the task's state. The scheduler checks expiry both when handling requests and during its periodic scan.

3. Safety

Original MS1 statement:

"A task will not be marked complete unless successful completion is reported, and a task will not run before its required dependencies finish. Retried tasks will not be recorded as multiple successful completions. These claims are violated if an unfinished task is marked complete, a task runs before its dependencies, or one logical task is recorded as completed multiple times."

The following properties must hold under the fault model in Section 5, even if an attempt expires and its task is run again.

S1. Completion requires an accepted success report

The scheduler marks a task SUCCEEDED only after accepting a SUCCESS report from the current holder of an unexpired lease. Every reported output must already exist in the artifact store under the attempt's own key prefix, with the specified length, SHA-256, and media type. Silence, expiry, or an existing file is not sufficient. A stale or expired report receives `409 LEASE_EXPIRED` or `409 CONFLICT`.

A `task_succeeded` event without an accepted report from the current owner would violate S1. So would success accepted after `lease_expired` for that attempt. The fault tests specifically exercise late reports after lost heartbeats, pauses, and partitions.

S2. Dependencies complete before execution

The scheduler assigns a task only after each parent has an accepted success. A worker begins the operation only after its start is acknowledged and uses the parents' accepted outputs. An assignment before a parent's `task_succeeded`, or a `worker_operation_started` without a matching `task_started` acknowledgment, would violate S2.

S3. One accepted completion per task

For each logical task `(job_id, task_id)`, the scheduler recognizes at most one active owner and accepts at most one success. If a worker repeats an accepted report, the scheduler returns the saved receipt and records `completion_duplicate`, rather than recording another success. Messages from superseded, failed, or expired attempts cannot change the accepted state. An exact replay of a stored failure receipt is also just a replay. The job is recorded as complete once, after all of its tasks succeed.

Two accepted successes, two owners recognized at the same time, or `job_completed` before all task successes would violate S3. This does not mean only one worker can physically execute the task. An expired worker may still be running while a new attempt starts. Its files use a different attempt prefix, and the scheduler rejects its late report. We check output integrity but do not recompute the worker's result.

Tests export scheduler and worker histories for checking by `check_history`. To check saved evidence directly, use `python -m tests.harness.check_evidence <dir>`. The checker verifies the recorded executions, not all possible executions. The precise test mapping for S1-S3 is in test-matrix.md.

4. Liveness

Original MS1 statement:

"If workers remain available, every ready task will eventually be assigned and either complete or be retried after failure. If a worker fails while executing a task, that unfinished task will eventually become available for reassignment. This claim is violated if an unfinished task remains permanently stuck."

MS2 intentionally left recovery from a crashed worker incomplete. MS3 adds leases to address that case. Both liveness properties depend on the availability and timing assumptions in Section 2.

L1. Ready tasks make progress

When a worker can communicate with the scheduler and artifact store, ready tasks are assigned in FIFO order. An attempt eventually succeeds, reports a failure, or loses its lease. Local failures and operations exceeding `OPERATION_TIMEOUT_MS` (default 30 seconds) produce failure reports. Failed or expired tasks return to the queue with a higher attempt number on the next assignment. A slow worker that continues to renew can retain its task.

A ready task left unassigned despite a polling worker, or an attempt that never ends under these conditions, would violate L1.

L2. Work held by failed workers is reassigned

A worker that crashes, pauses, or stops renewing loses its attempt when the lease expires. The scheduler scans for expiry every `LEASE_SCAN_MS` (default 100 ms), requeues the task, and allows the next available worker to claim it. An unfinished task permanently held by a failed worker violates L2. The bounded-recovery measurement, P-REC, is defined in Section 6.

Progress is not guaranteed if every worker is unavailable, a partition is permanent, or the scheduler is down. A permanently failing operation will be retried without limit, leaving its job RUNNING. A live worker may also lose its lease when heartbeat requests are delayed. If a link delivers renewals but permanently loses every completion report, the fair-loss assumption is violated and progress may stop.

5. Fault model

Original MS1 statement:

"The system will handle worker crashes, worker restarts, temporary worker unavailability, and delayed or lost communication. Byzantine or malicious worker behavior is outside the scope of the project. These failures will be tested by stopping workers and introducing communication problems during active jobs."

The covered failures are worker crashes, restarts, pauses, slow execution, and temporary worker-to-scheduler communication problems. Crash tests stop workers after a claim, during execution, after upload, and after reporting. The scheduler reassigns unfinished work after expiry, and results from old attempts cannot be accepted. A restarted worker gets a new session.

For lost, delayed, duplicated, or reordered messages, workers retry with the same request identifiers and use stored receipts where applicable. A short pause or partition need not cause expiry if the worker renews in time. A client can also retry the same job ID and identical manifest; reusing the ID with different contents returns `409`.

The harness uses SIGKILL, SIGSTOP/SIGCONT, Docker kill/pause, forced pod deletion on the course cluster, test-only worker stall points, and a per-worker proxy for message faults. It records the injected fault alongside each run's history. See test-matrix.md for the individual scenarios.

These cases are not covered by the recovery guarantee:

- Scheduler crashes or restarts. A new run starts with empty state, and old-run requests receive `409 STALE_RUN`. The tests check stale-run rejection, not durable recovery.
- Artifact-store crashes or data loss. Temporary unavailability is partly handled: submissions return `503` without acceptance and unverifiable success reports can be retried. The store itself is not recovered.
- Injected partitions involving the artifact store. Its availability and correctness remain assumptions; short outages may be absorbed by retries.
- Malicious or Byzantine workers, forged identities, false computations, and clock tampering.

6. Performance

Original MS1 statement:

"We will measure job completion time, task throughput, scheduling latency, and recovery time after worker failures. Performance will be evaluated by submitting increasing numbers of concurrent DAG-based jobs under normal operation and while intentionally introducing worker failures."

Job completion time runs from acceptance to completion. Throughput counts accepted task successes per second. Scheduling latency is the time from READY to ASSIGNED. Recovery time runs from the lost attempt's last accepted renewal to assignment to a new worker; it includes detection and waiting in the ready queue. It is not measured from the external worker-kill command.

The benchmark runs six-task jobs using 100 ms fixture operations, with four warmup jobs and 24 measured jobs per run. `make bench` runs the MS2 fault-free matrix for W = 1, 2, and 4 workers. `make bench-recovery` fixes W = 4 and runs at C = 1, 4, and 16 concurrent jobs, comparing fault-free deployments with runs that kill and replace busy workers. Each service is limited to one CPU and 512 MiB.

Load is set by C, W and task duration. In the MS2 fault-free matrix, throughput grew with W, about 4.2 tasks/s per one-slot worker; at C = 16 with four workers we could not tell what limited throughput (progress report, Section E). The scheduler's own load grows with W even when there is no work: every request takes its single mutex, each idle worker claims every `POLL_INTERVAL_MS`, and each busy worker renews once a second. A large enough fleet would therefore saturate the scheduler, but we have not measured where. There is no admission control: work beyond capacity waits in the FIFO ready queue and job time grows, up to the memory estimate in Section 1.

P-REC. Bounded recovery with an idle worker

An attempt must not expire before `LEASE_TIMEOUT_MS` has passed since its last accepted renewal. With an idle worker polling, reassignment must happen within `LEASE_TIMEOUT_MS` plus two seconds. Without an idle worker, queueing adds delay. The bound does not cover arbitrary network delays or scheduler pauses.

The measurements, incomplete run, and hardware limitations are reported in Section E of the progress report. Raw results are in `results/ms3-logs/recovery-benchmark/`.

7. Configuration that changes behavior

- `LEASE_TIMEOUT_MS` (scheduler, 5000 ms): maximum time without a renewal.
- `LEASE_SCAN_MS` (scheduler, 100 ms): background expiry-scan interval.
- `HEARTBEAT_INTERVAL_MS` (worker, lease/5 from `/v1/health`): heartbeat spacing.
- `POLL_INTERVAL_MS` (worker, 100 ms): interval between idle claims.
- `OPERATION_TIMEOUT_MS` (worker, 30000 ms): operation timeout before failure.
- `ENABLE_TEST_HOOKS` (worker, false): test-only stall and failure controls; leave disabled outside tests.

8. Resources and replication

`make test` runs the container-based test suite. The host needs Docker, `make`, and a POSIX shell; the container supplies the language tools. The reference benchmark host was a Windows 11 laptop with 12 CPUs, 16 GB RAM, Docker Desktop 27.1.1, and a Linux VM capped at 3 GB. Compose services were

limited to one CPU and 512 MiB each. The smoke test, the functional workload and both worker-crash tests also run on the course Hokea cluster; see `deploy/hokea/README.md`.

The repository contains a five-task arithmetic DAG (expected result 22), a six-task benchmark DAG (expected result 9), and a reproducible ten-second 1280 x 720 synthetic video with fixture subtitles.

9. Changes since MS2

MS3 replaces the intentionally broken worker-crash recovery behavior with leases, heartbeats at `POST /v1/attempts/heartbeat`, command-time expiry checks, and a periodic scan. Expired attempts cannot report success. The original MS2 crash-reassignment test is now expected to pass.

We also clarified S1 to require a current lease holder, distinguished S3's single accepted completion from repeated execution, and made the tested fault model and recovery bound explicit. After the first MS3 submission we fixed the Hokea adapter, which had rejected the course runner's Hokea build, and added the crash test requested in the MS2 feedback; both crash tests also pass on the course cluster. The results are in the progress report; the exact tests are in the test matrix.